115-1 TAICA《生成式 AI：文字與圖像生成的原理與實務》

07 讓 AI 看懂你的世界

Context Engineering 與 RAG

政治大學應用數學系 蔡炎龍

01.

RAG

RAG: 一個降低錯誤的方法

再度強調理解原理、
不要被大字彙迷惑。

RAG

Retrieval-**A**ugmented **G**eneration

引發非常多的討論

FUTURE COMMERCE 未來商務
找文章 AI 解方 找活動
找文章 > TRENDS
RAG：讓 AI 更聰明的秘密武器，揭開生成式 AI 的新篇章
2024.10.28 | 偉利科技執行長 黃適文

businesswire A BERKSHIRE HATHAWAY COMPANY
NetApp與NVIDIA合作重新定義企業RAG並為代理式AI提供支持
NTAP資料基礎架構與強大的NVIDIA NeMo Retriever與NIM微服務相雲端中發現、搜尋和管理資料的方式，為AI應用提供助力
放了跨雲端和本地基礎架構儲存的EB級企業資料，以驅動RAG功能，使企業的用程式。
、NetApp ONTAP和NetApp BlueXP統一控制平面，以及NVIDIA NeMo軟體平台中)。

資策會Find
輕鬆與汽車對話！賓士運用「RAG」生成式AI技術打造「有話直說」功能
其工作原理簡單來說，就是當你向系統提問時，RAG會先在資料庫或其他外部數據源中檢索出與問題相關的資料。這些資料會被轉換成數位化的嵌入向量，代表資料的...

遠見
最新 話題 國際 金融 AI 科技 產經 教育 民調
首頁 > AI > 時事新知
要當生成式AI新創，RAG就是最佳舞台
郭家齊 2024-06-07
分享
收藏

T客邦
什麼是RAG？RAG技術突破LLM限制，結合「資訊檢索」和「文字生成」動態獲取知識大幅提升
RAG是一種全新的LLM訓練模式，透過將資訊檢索與文字生成結合，讓LLM能夠動態取用並理解新的知識，提升其在特定場景下的問答及文字生成品質。

網管人
避免AI幻覺RAG漸興起 防外洩「私有」當道
避免AI幻覺RAG漸興起防外洩「私有」當道 ... 企業對人工智慧與GenAI的態度，已經從觀望評估逐步走向在生產環境的落地應用。基於機敏資料的安全、人才以及成本...

奇摩股市
Vespa.ai宣布在檢索增強生成（RAG）中支援ColPali
ColPali將包括視覺元素在內的整個彩現文件嵌入到專為大型語言模型（LLM）最佳化的向量表示中，從而增強了文件檢索功能。這樣便能夠減少延遲，提高準確性，並...

INSIDE
RAG 是什麼？就像是大型語言模型的「小抄」
RAG (Retrieval-Augmented Generation) 是一種自然語言處理模型，它結合了檢索（retrieval）和生成（generation）的技術。它使用了檢索模組來從大量資料中...

其實就是討論 Content Engineering

可以自己放、但不能叫電腦找嗎?

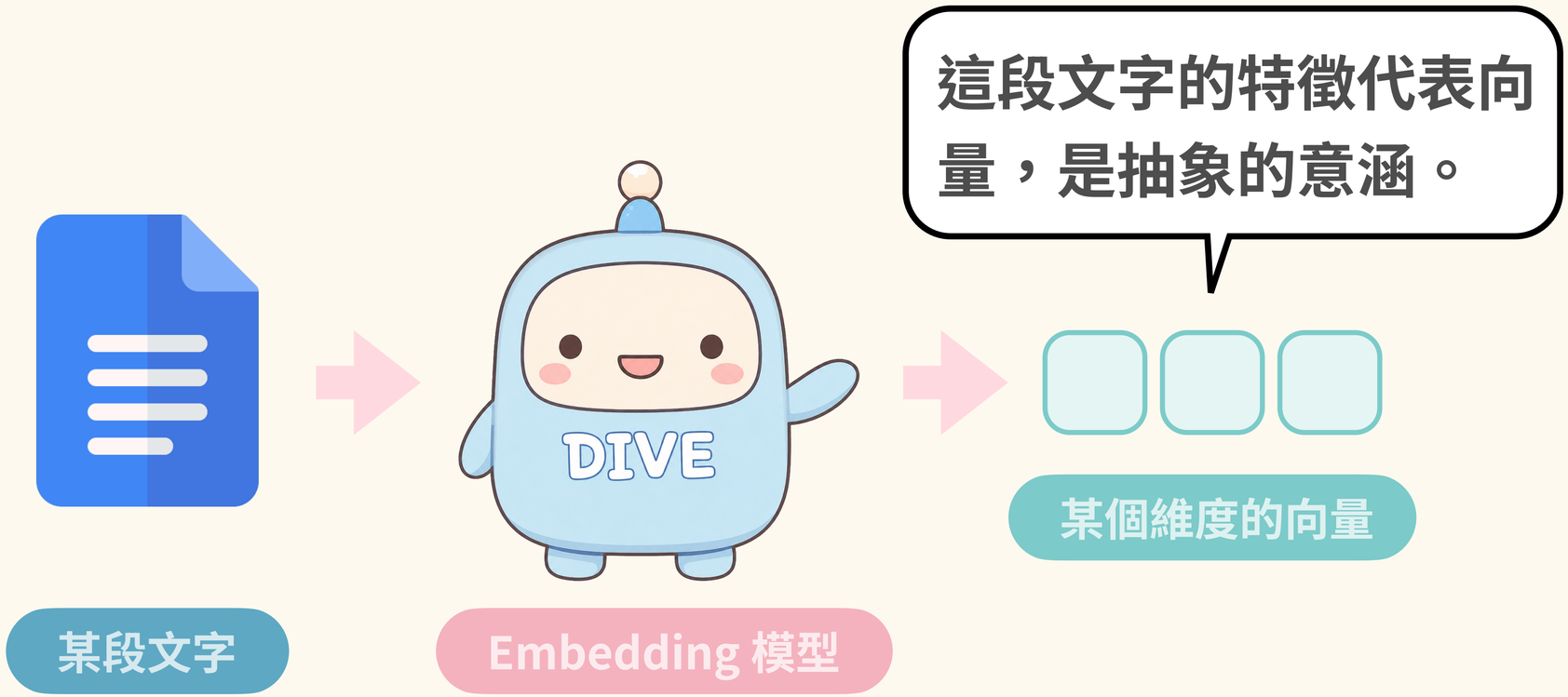
正確的資訊

提供需要的正確資訊。

清楚的指引

例如, 以上面的資訊, 用什麼樣的格式、風格, 來回答使用者的問題。

重點是我們要找一個 embedding 模型



基本上就是一個語言模型

有時會用對比學習 Contrastive Learning

寫成數學式子像這樣

$$\mathscr{L}(\theta) = -\log \frac{e^{\text{sim}(\mathbf{q},\mathbf{k}^+)}}{\sum e^{\text{sim}(\mathbf{q},\mathbf{k})}}$$

有時會區分「問題」和「內容」

query:

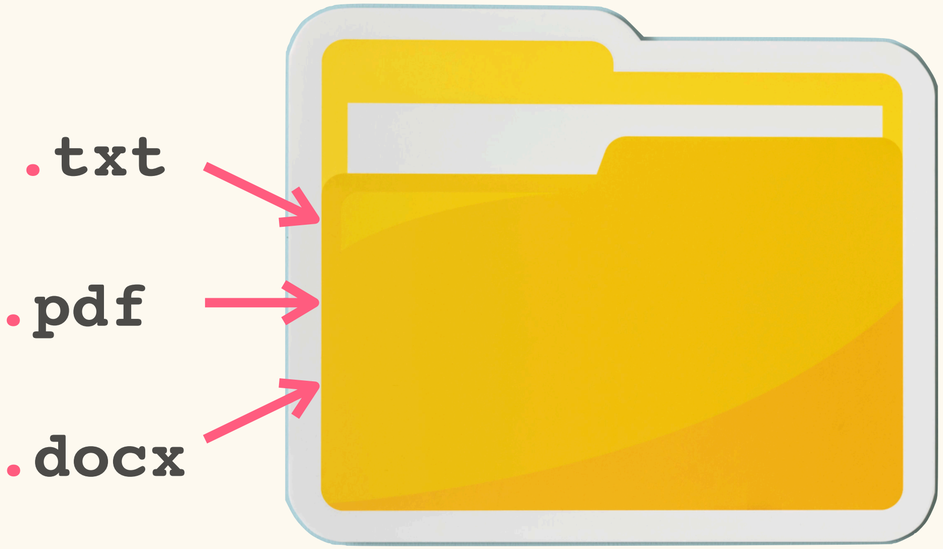
圖書館星期日幾點關門？

passage:

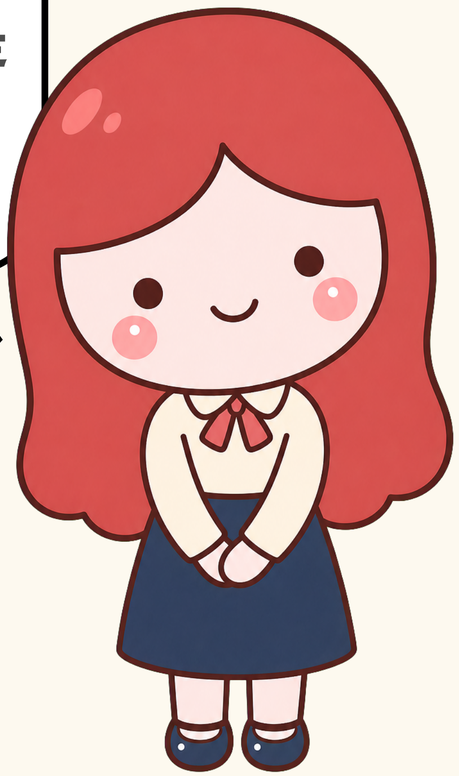
本館週日開放至下午五點。

要注意這種 prefix 沒有通用寫法，有些模型完全沒有使用。

最傳統無腦做法是把所有的檔案送一個資料夾



基本上**純文字檔案**都沒問題, 但其實 PDF 或是 Word 檔也都可以!



最傳統 RAG 就是自動切成一個一個文字塊

500 字

重疊 50 字

500 字

基本上要考慮多少字是一個「文字塊」，還有需不需要有重疊。

其實重點還很多...

可能的一些考量

500 字

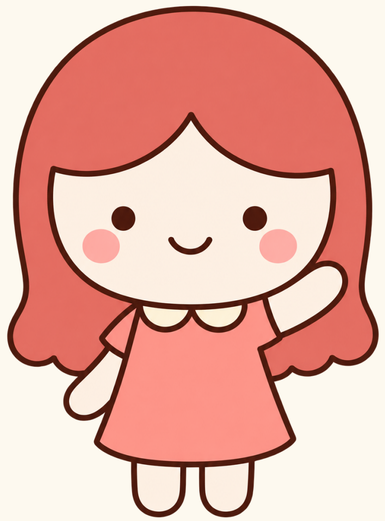
重疊 50 字

500 字

文字塊大小、要怎麼切，像法修是否要一條一條切? 還有重疊要多少字?

比較新的作法是自己切成自己想要的樣子!

做 embedding 的部份



標題：部分工時的定義

資料類型：官方資訊的教學改寫摘要（非網站全文、非法律意見）
主管機關：中華民國勞動部
來源網址：https://www.mol.gov.tw/1607/28690/87756/90011/90021/
教材整理日期：2026-10-18

metadata

【內容】

部分工時，是指約定工作時間較事業單位全時勞工明顯縮短的工作安排，縮短的時數可由勞雇雙方議定。工讀、兼職或假日工作，並不代表勞動權益自然降低；是否適用勞動法規，仍須考量實際僱傭關係與事業適用情況。

【檢索練習問題】

我只是週末工讀生，還有勞基法保障嗎？

【提醒】

這是為 RAG 教學重寫的簡短摘要；完整適用條件、例外及最新修訂，請開啟來源網址核對。

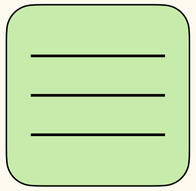
實際上有兩個部份

真的要做 RAG (Embedding) 的文字

metadata

可以做關鍵字搜尋。

Metadata: 文字塊的附加資訊



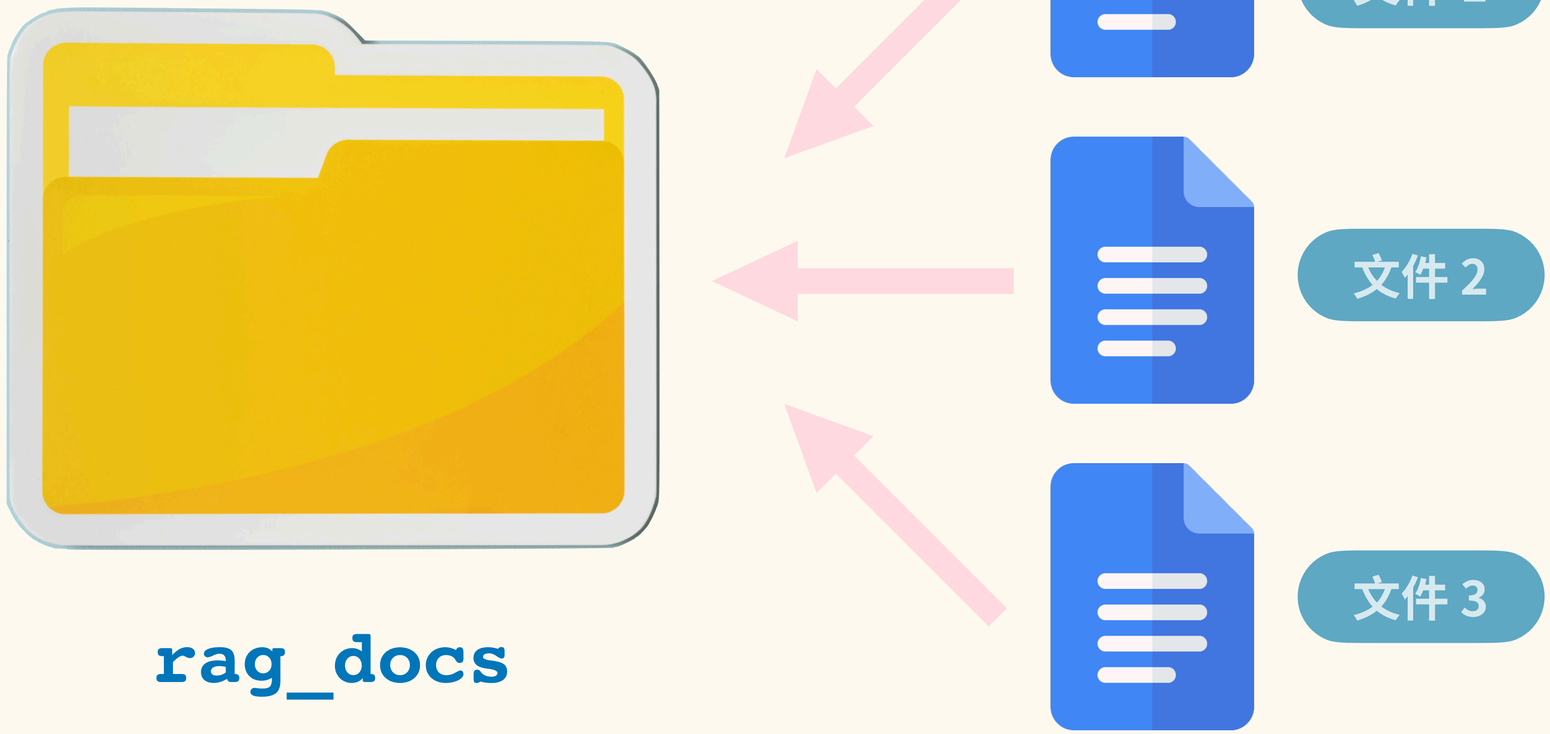
文字塊

比如這段文字的來源 (原文)、所在頁數、文章標題、作者、日期、分類...等等。

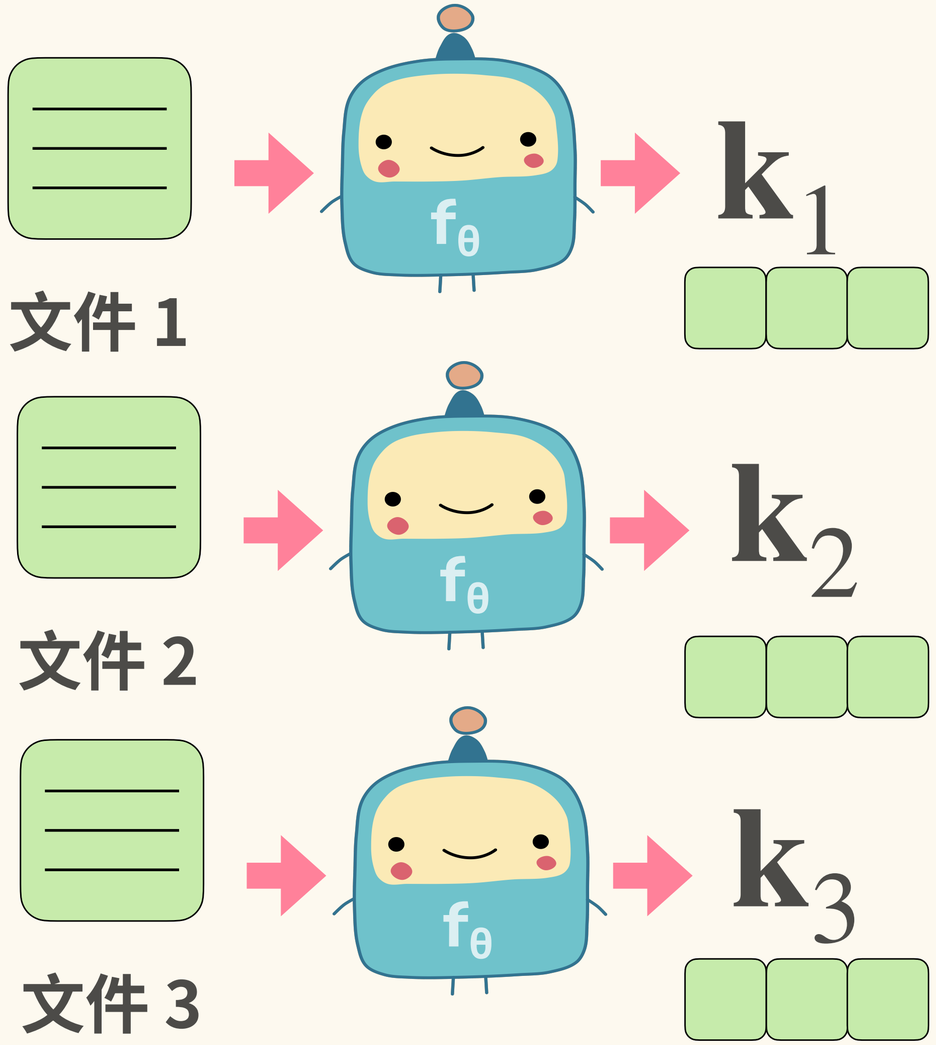
需要 Metadata 的例子

我想知道什麼是 AI Agents，但不用管炎龍老師怎麼說，因為他都亂講。

然後把你整理好的文件放進一個資料夾



每段文字就給它一個代表向量

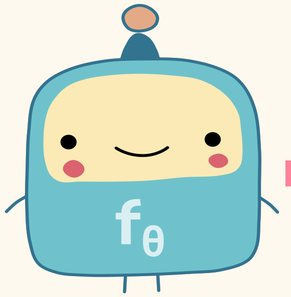


每份重要資料都找到自己的**特徵代表向量**。

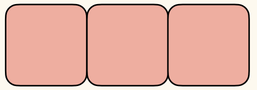
在我們下 prompt 問問題時

問題也化為特徵
代表向量。

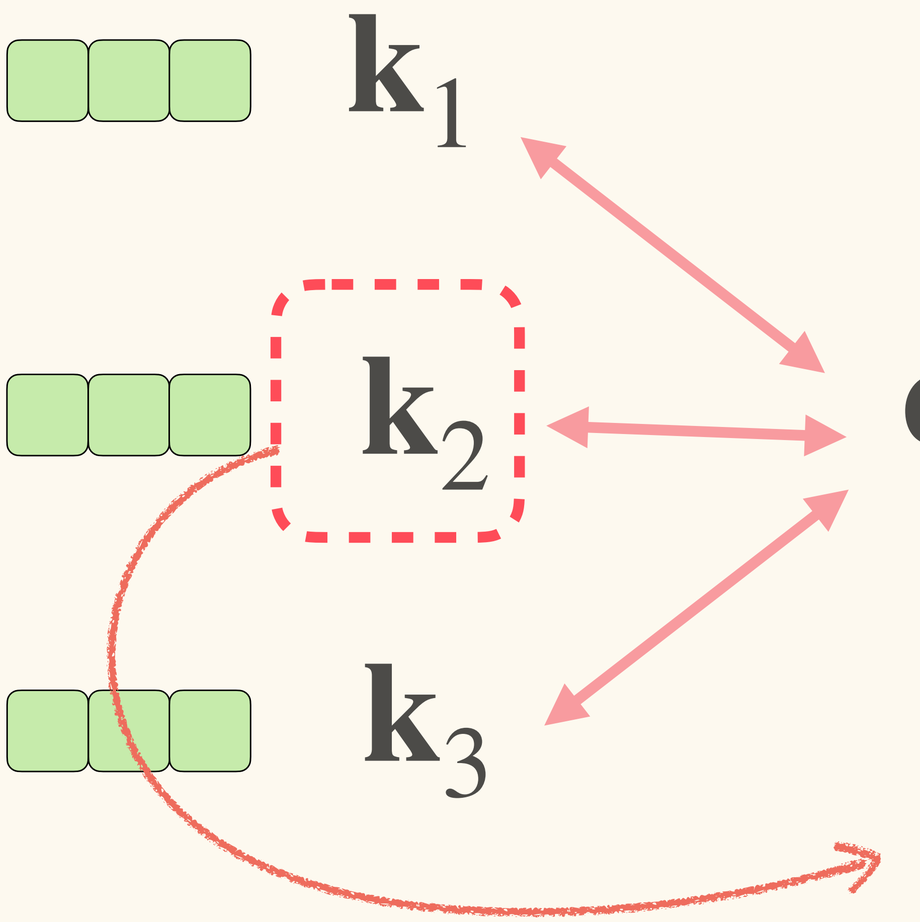
一個問題 (prompt)



$\mathbf{q}$



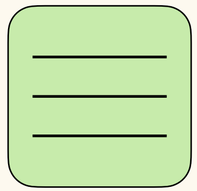
比較和誰最接近，一起放入 prompt 中



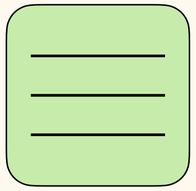
這是近來 AI 搜尋方法!

和原本問題最接近、最相關的資料。

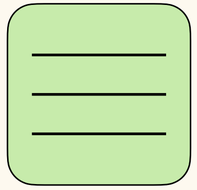
找出的資料通常不會只有一筆！



文字塊 1



文字塊 2



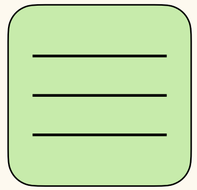
文字塊 3

有可能需要重新排序!

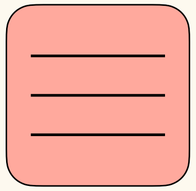
Reranking

需要加入上下文或相關文章嗎？

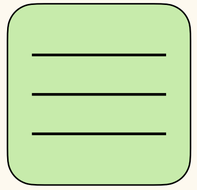
1



前一塊文字

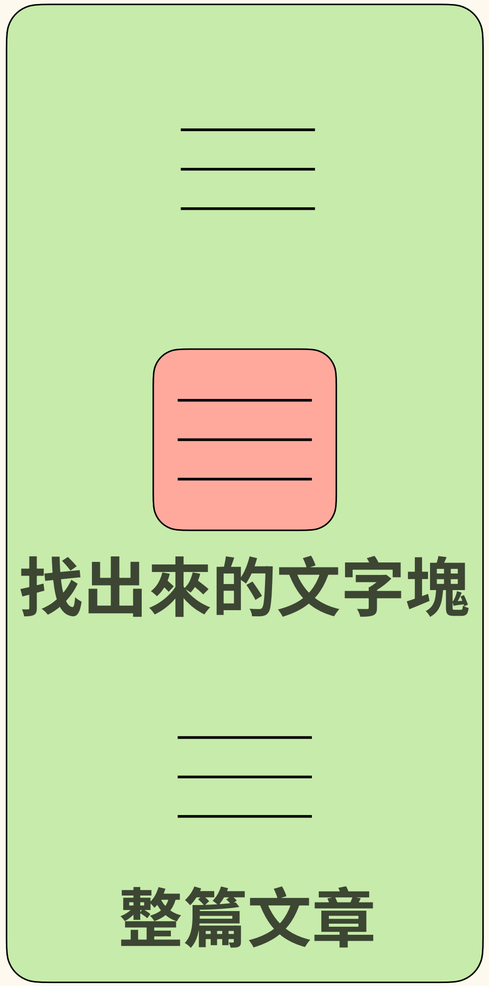


找出來的文字塊

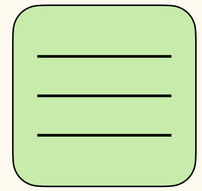


後一塊文字

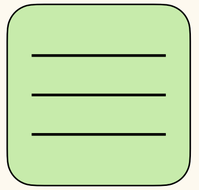
2



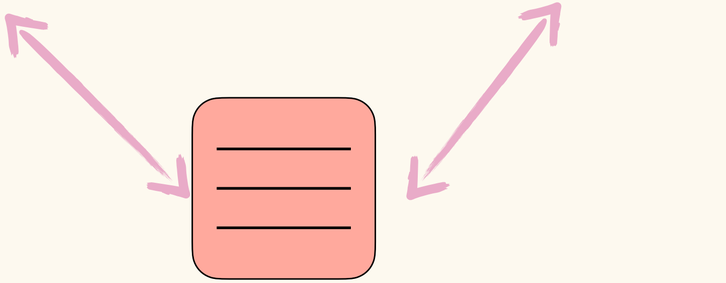
3



相關文章1



相關文章2



找出來的文字塊

基本上會有這些資訊

`question`

使用者的問題 (原始 prompt)。

`retrieved_chunks`

用 RAG 找到的資訊。

接著就設計新的 prompt

例如寫這樣的 prompt。

“請根據 `{retriever_chunks}` 裡的資訊，
來回應使用者的問題: `{question}`”

基本上就是現在火紅 RAG 的概念

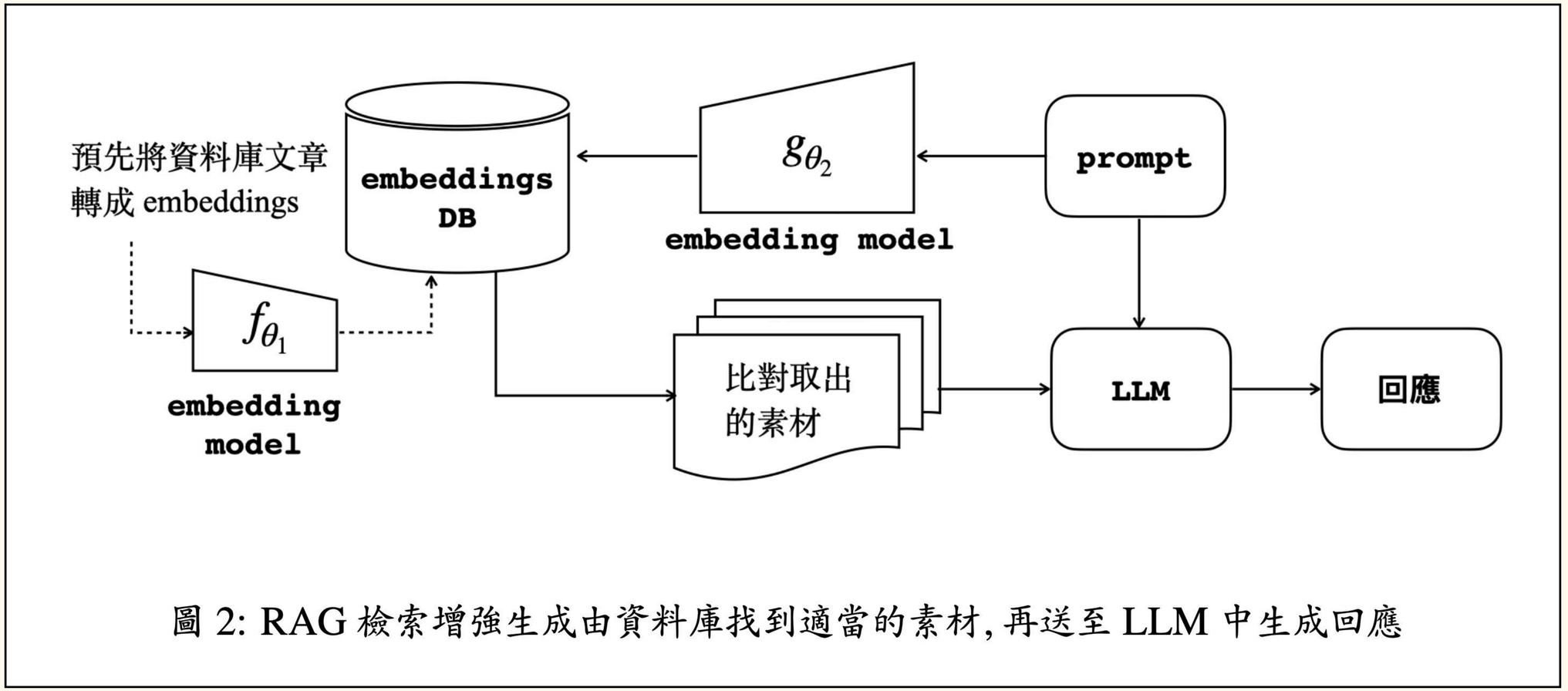


圖 2: RAG 檢索增強生成由資料庫找到適當的素材, 再送至 LLM 中生成回應

話說回來: 真的要 RAG 嗎?

全部的資訊都放 Prompt
不就結了嗎?

特別是語言模型上下文越來越長...

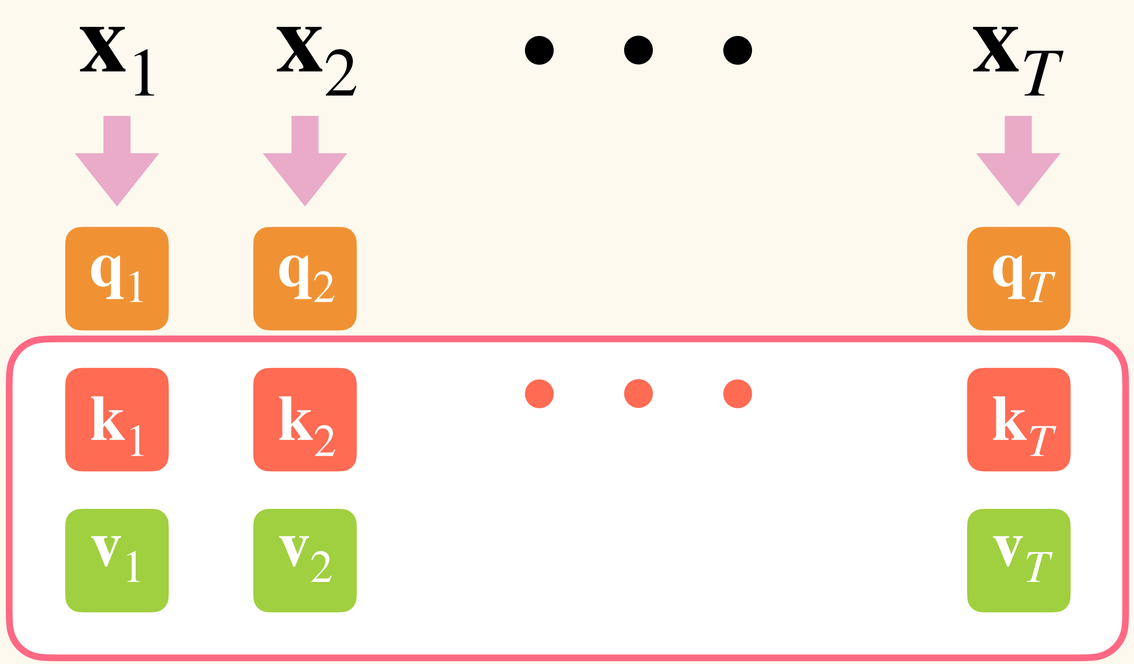
CAG: 用 KV-Cache 技術把資訊都塞給 LLM

Don't Do RAG:
When Cache-Augmented Generation is All You Need for Knowledge Tasks

Brian J Chan*
Chao-Ting Chen*
Jui-Hung Cheng*
Department of Computer Science
National Chengchi University
Taipei, Taiwan
{110703065,110703038,110703007}@nccu.edu.tw

Hen-Hsen Huang
Insititue of Information Science
Academia Sinica
Taipei, Taiwan
hhhuang@iis.sinica.edu.tw

鍵值緩存 (KV-Cache)



02.

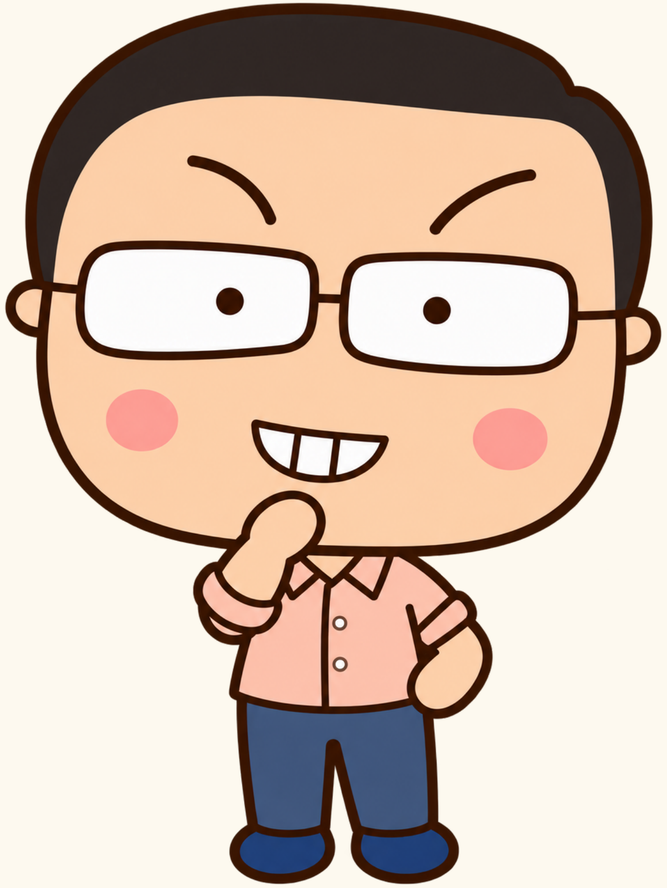
記憶的一種方式

同一個系統，可以用兩個以上的向量資料庫

向量資料庫 1

向量資料庫 2

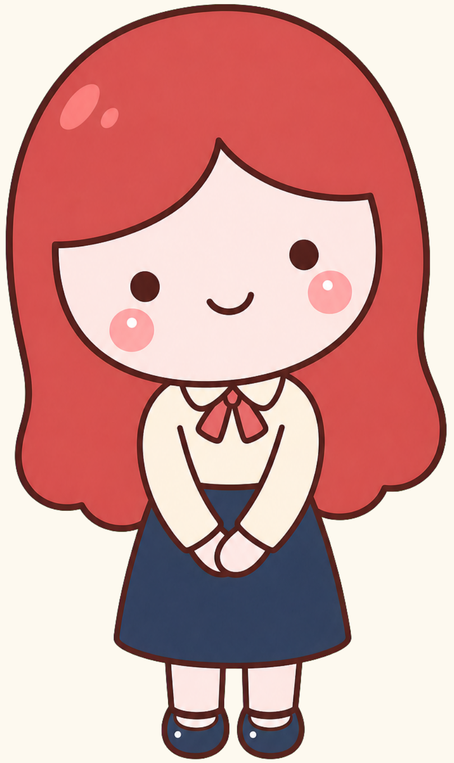
接著教大家很嘴的事



我們在外面會聽到有人說, 大型語言模型的「記憶」有兩種: **短期記憶**和**長期記憶**。

大型語言模型的「短期記憶」

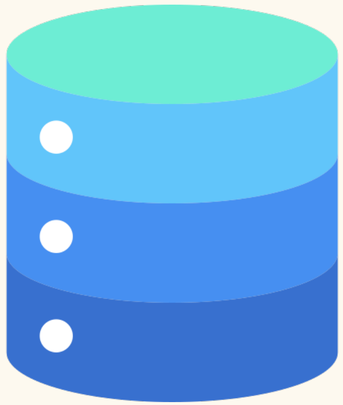
就是我們這次和大型語言模型的對話過程。



下次來, 這次說的話都忘了。
更糟糕的是, 對話過程太長,
前面的話也會忘了。

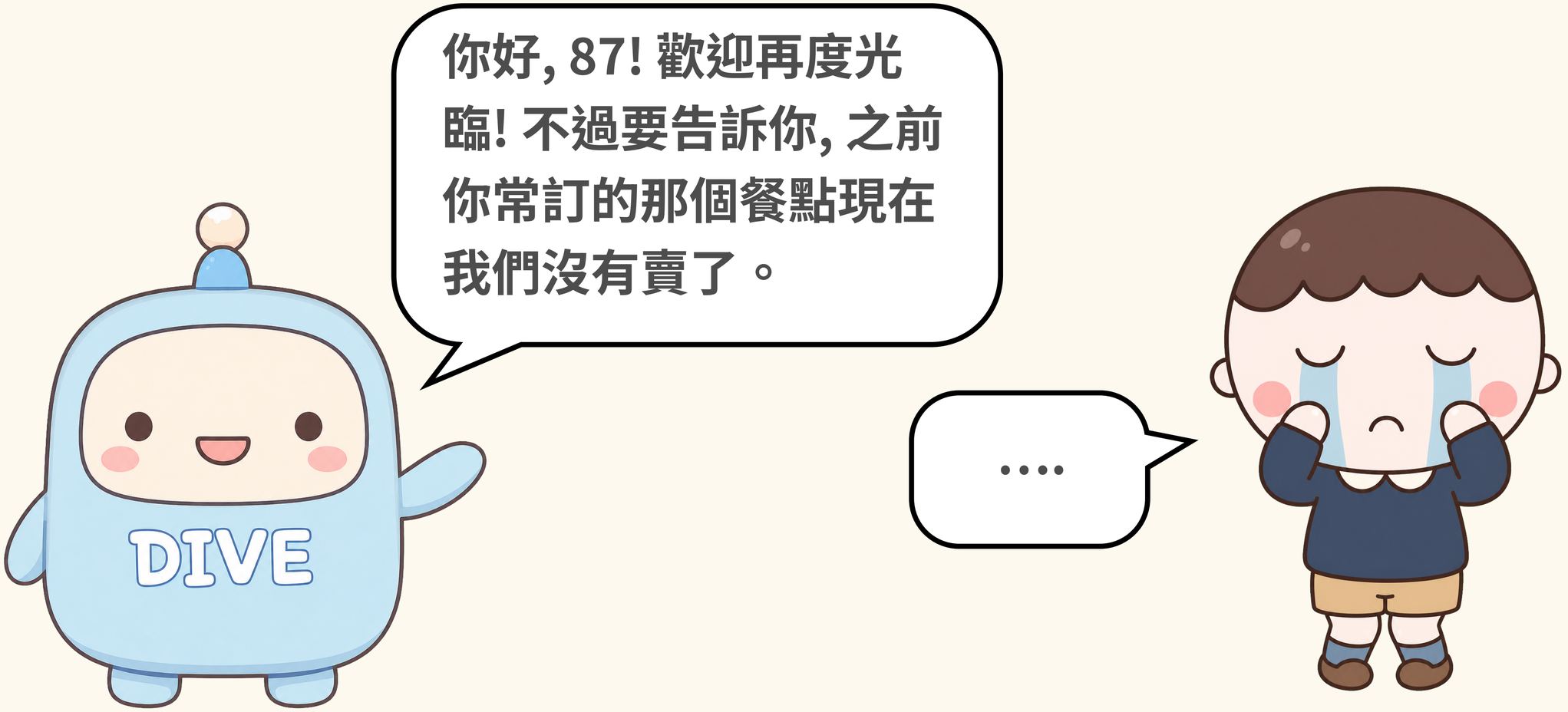
怎麼做大型語言的「長期記憶」呢?

一個方法就是做 RAG。



過去完整的對話,
做成向量資料庫

特別是客製化的對話機器人，會記得你的點點滴滴



03.

LLM Wiki

2026 年 4 月 Karpathy 引爆的另一篇貼文

Andrej Karpathy
@karpathy

LLM Knowledge Bases

LLM 知識庫

Something I'm finding very useful recently: using LLMs to build personal knowledge bases for various topics of research interest. In this way, a large fraction of my recent token throughput is going less into manipulating code, and more into manipulating knowledge (stored as markdown and images). The latest LLMs are quite good at it. So:

最近我發現非常有用的一件事：使用 LLM 來建立個人知識庫，以各種研究興趣的主題為主。這樣一來，我近期的大部分代碼操作量就減少了，而更多的是進行知識操作（以 markdown 和圖片的形式存儲）。最新的 LLM 在這方面表現得相當出色。因此：

簡單說，是打造你自己的 Wikipedia

教 AI 怎麼把檔案
產生/修改 wiki 檔

你找的資料

raw

wiki

`index.md`

Karpathy 原文翻譯

最近我發現一件非常實用的事：運用 LLMs，針對各種感興趣的研究主題建立個人知識庫。這麼一來，我最近大量的 token 用量便不再那麼多用於操作程式碼，而是更多用於整理知識（以 Markdown 和圖片的形式儲存）。最新的 LLMs 在這方面表現相當出色。所以：

資料匯入 Data Ingest

資料匯入

我會將來源文件（文章、論文、程式碼儲存庫、資料集、圖片等）索引到 raw/ 目錄，接著使用 LLM 逐步「編譯」出一個 Wiki，也就是存放在目錄結構中的一系列 .md 檔案。Wiki 會包含 raw/ 中所有資料的摘要與反向連結，接著將資料分類為各種概念，為這些概念撰寫文章，並將它們全部串連起來。若要將網頁文章轉成 .md 檔案，我喜歡使用 Obsidian Web Clipper 擴充功能；另外也會用快速鍵將所有相關圖片下載到本機，讓我的 LLM 能輕鬆參照。

IDE

IDE

我使用 Obsidian 作為 IDE 的「前端」，在裡面檢視原始資料、編譯完成的 Wiki，以及衍生出的視覺化內容。值得注意的是，Wiki 中所有資料都是由 LLM 撰寫並維護，我很少直接動手修改。我也試用了幾款 Obsidian 外掛，看看能不能用其他方式呈現及檢視資料（例如用 Marp 製作投影片）。

Q&A

Q&A

有趣的地方在於，當你的 Wiki 夠大時（例如我最近某項研究的 Wiki 有 ~100 篇文章、~400K 個字），你就可以向 LLM 代理詢問各種複雜問題，讓它自行研究答案等等。我原本以為得動用複雜的 RAG，但 LLM 在自動維護索引檔和所有文件的簡短摘要方面表現相當不錯，而且在這種 ~小規模下，也能輕鬆讀取所有重要的相關資料。

輸出 Output

輸出

比起在文字或終端機中取得答案，我更喜歡讓它為我產生 Markdown 檔案、投影片（Marp 格式）或 matplotlib 圖片，然後再用 Obsidian 檢視。你可以想像，視查詢內容而定，還有許多其他視覺化輸出格式。我常常會把輸出結果「歸檔」回 Wiki，讓它能在之後的查詢中派上用場。這樣一來，我自己的探索和查詢就會不斷累積到知識庫中。

檢查 Linting

檢查

我用 LLM 對 Wiki 執行過一些「健檢」，例如找出不一致的資料、透過網路搜尋補齊缺漏資料、找出有趣的關聯以做為新文章的候選題材等等，藉此逐步整理 Wiki，並提升整體資料的完整性。LLMs 很擅長建議值得進一步提出和探究的問題。

額外工具 Extra tools

額外工具

我發現自己會開發額外的工具來處理資料。例如，我用 vibe coding 的方式，為 Wiki 寫了一個簡單又陽春的搜尋引擎。我會直接使用它（透過網頁介面），但更多時候，我想透過 CLI 把它交給 LLM，作為處理大型查詢的工具。

進一步探索 Further Explorations

進一步探索

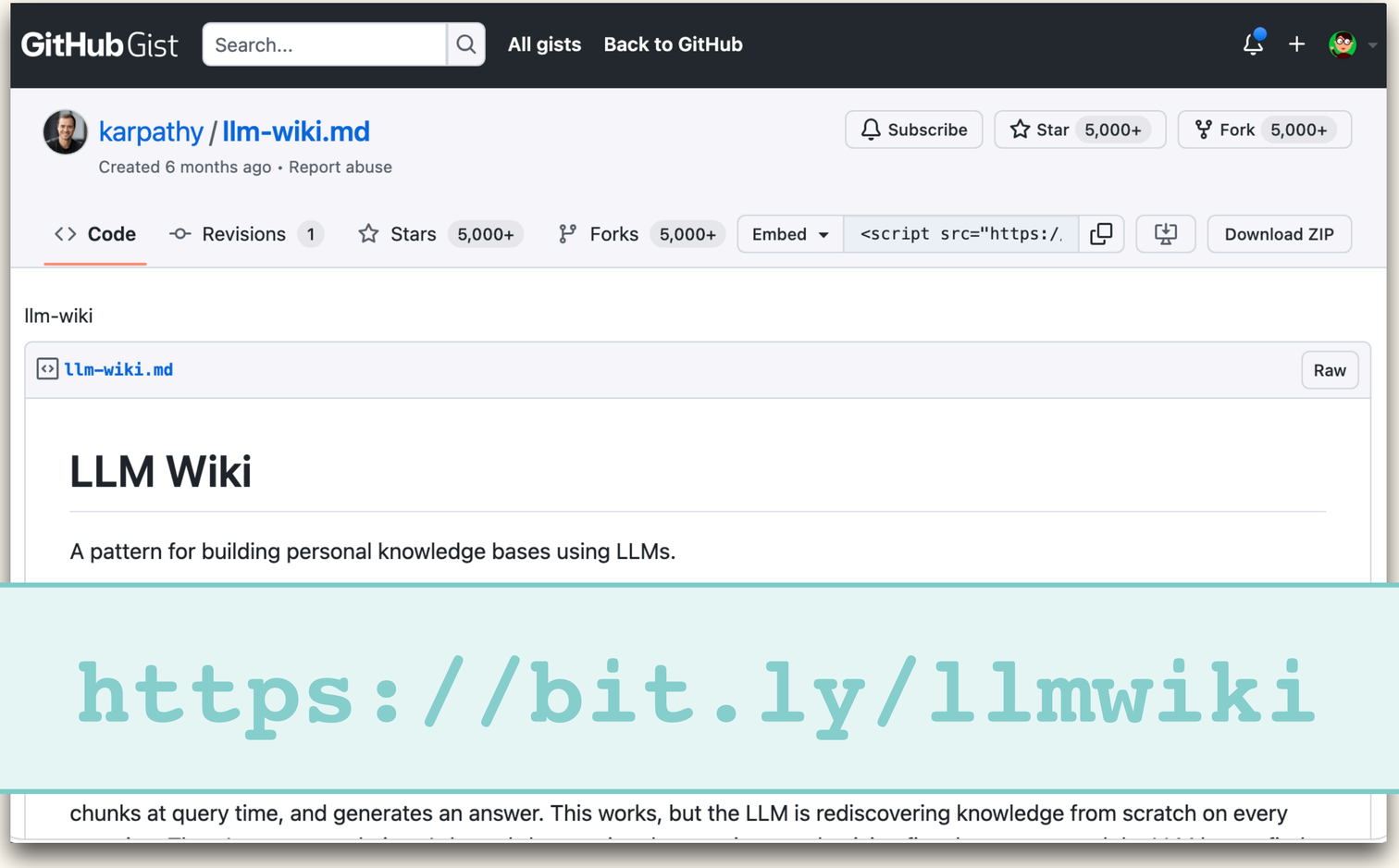
隨著資料庫逐漸擴大，自然會想進一步思考如何生成合成資料並微調模型，讓你的 LLM 把資料「記」在權重裡，而不只是依賴上下文視窗。

懶人包 TLDR

懶人包

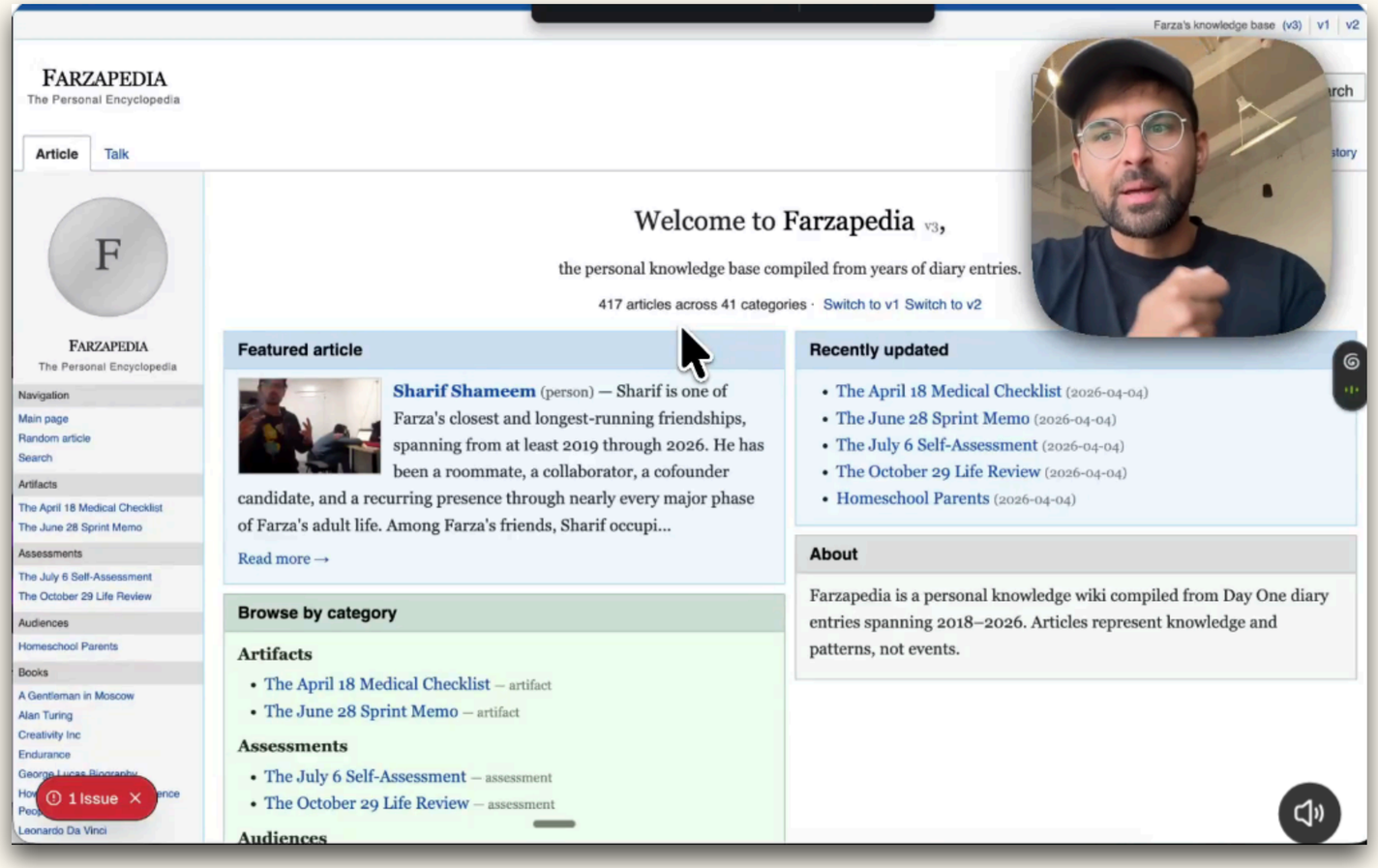
收集一定數量來源的原始資料，再由 LLM 整理成 .md 維基，接著讓 LLM 透過各種 CLI 工具操作維基，進行問答並逐步擴充內容，所有資料都能在 Obsidian 中檢視。你幾乎不會手動撰寫或編輯維基，這是 LLM 的領域。我認為這裡有機會打造出令人驚豔的新產品，而不是一堆東拼西湊的指令碼。

Karpathy Gist 分享 LLM



https://bit.ly/llmwiki

Farzapedia：個人化的 wikipedia



04.
【作業】打造你的 RAG 系統

打造自己的 RAG 強化對話機器人!

程式 1 用自己的資料、建立向量資料庫

`https://yenlung.me/rag01colab`

程式 2 實作 RAG 系統

`https://yenlung.me/rag02colab`

作業：打造你的 RAG 系統

- 可以找一個你有興趣的文件 (比如說某個規定、相機說明書等等)
- 用 Vibe Coding，把你的數據，打包成一個個的檔案，存成，比如說 `rag-data.zip`。
- RAG 01: 用自己的數據，打造向量資料庫，並打包你的向量資量庫，如 `vector_embedding.zip`。
- RAG 02: 程式要能讀入你的 `vector_embedding.zip` 並解壓縮。
- 打造自己的 RAG 強化的對話機器人。

Q&A